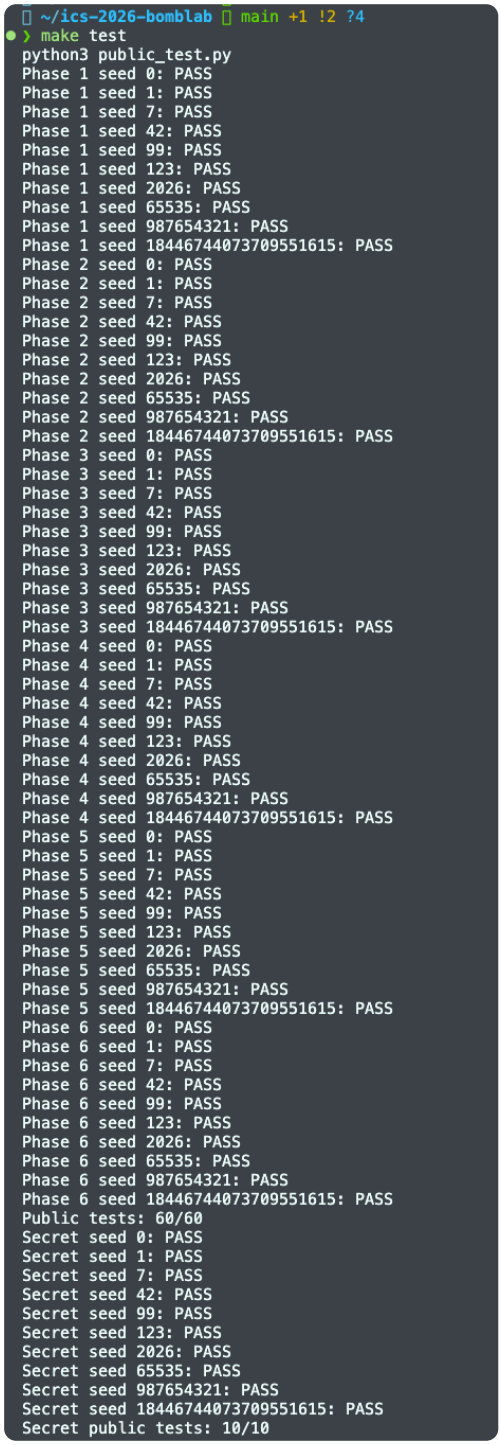
Lab2 BombLab 实验报告

运行结果

公开测试



六关和 Honor 的公开测试全部通过。各关在公开列表以外的 seed（114514）上的验证记录附在每一节的末尾。

通用工具与方法

我们通过 phase0 的学习，发现 bomb 的汇编代码干的事情基本就是 input challenge 和一行输入，这个一行通过 `sscanf` 输入到数组，然后比较一下从 challenge 计算的值和几个数组的是否相等。我们的任务就是恢复出这个计算关系，输出数组组成的这一行（secret phase 有些许不同）。

因此我们需要先 dump 汇编代码，对于 phase1–6 来说直接递归 dump 出来就行了，

```
objdump -d --no-show-raw-insn --disassemble=phase_01 bomb > phase_01.S
```

用上面这种就能找到每个函数对应的汇编，这个 lab 大部分情况我只看汇编就行了，较少用到在线调试。

然后类似通用的，我们先要确定 count（输入数组的数量），所有 phase 都有 `sscanf` 这个函数，我们只要看他返回值后面的比较就行了，例如

```
4020d3:  cmp    $0x1,%eax
```

每个 phase 都有类似的，后面不赘述。然后后面有些纯计算逻辑的地方我也省略了，主要抽象出一些考点比如调用、循环、递归。

Phase 1

phase 1 比较简单 `rsi` 被放到 `r12`，然后 `r12` 被放到 `rdi` 传入 `p1_compute`，然后调用如下

```
0000000000402050 <p1_compute>:
  402050:  endbr64
  402054:  push   %rbp
  402055:  mov    %rdi,%rax
  402058:  mov    %rdi,%rdx
  40205b:  mov    %edi,%r8d          // r8d = low 32 bits of challenge
  40205e:  shr    $0x20,%rax         // rax = challenge >> 32
  402062:  shr    $0x10,%rdx         // rdx = challenge >> 16
  402066:  mov    %eax,%r9d          // r9d = low 32 bits of (challenge >> 32)
  402069:  mov    %eax,%esi          // esi = low 32 bits of (challenge >> 32)
  40206b:  mov    %rsp,%rbp
  40206e:  call   402020 <p1_step0>
                                     // eax is now 32bits(challenge + high_32_bits(challenge) * 8) ^
0xb2c
  402073:  mov    %rdx,%rsi          // rsi = challenge >> 16
  402076:  mov    %r9d,%edi          // edi = low 32 bits of (challenge >> 32)
  402079:  mov    %eax,%ecx          // ecx = eax above
  40207b:  call   402030 <p1_step1>
                                     // (high32(challenge) - 32(challenge >> 16)) * 5 + 0x59cc, then
cut to 32 bits
  402080:  mov    %r8d,%esi          // esi = low 32 bits of challenge
  402083:  mov    %eax,%edi          // edi = eax above
```

```
  402085:  call   402040 <p1_step2>
  40208a:  pop    %rbp
  40208b:  xor    %ecx,%eax          // returns output of step2 ^ output of step0
  40208d:  ret
  40208e:  xchg   %ax,%ax
```

```
0000000000402020 <p1_step0>:
  402020:  endbr64
  402024:  lea    (%rdi,%rsi,8),%eax
  402027:  xor    $0xb2c,%eax
  40202c:  ret
  40202d:  nopl   (%rax)

// step0 returns (challenge + high_32_bits(challenge) * 8) ^ 0xb2c
```

```
0000000000402030 <p1_step1>:
  402030:  endbr64
  402034:  sub    %esi,%edi          // edi = edi - esi, which is high_32_bits(challenge) -
low_32_bits(challenge >> 16)
  402036:  lea    0x59cc(%rdi,%rdi,4),%eax
  40203d:  ret
  40203e:  xchg   %ax,%ax

// step1 returns (high32(challenge) - 32(challenge >> 16)) * 5 + 0x59cc, then cut to 32 bits
```

```
0000000000402040 <p1_step2>:
  402040:  endbr64
  402044:  lea    (%rdi,%rdi,2),%eax
  402047:  add    %esi,%eax
  402049:  xor    $0x24bb,%eax
  40204e:  ret
  40204f:  nop

// step2 uses edi extended to 64bits
// returns (rdi + rdi * 2 + esi) ^ 0x24bb -> b.
```

直接翻译出来就行了，没啥好说的，

```c
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    (void)challenge;
    // TODO: 根据 phase_01 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    count = 1;
    // step0 32bits(challenge + high_32_bits(challenge) * 8) ^ 0xb2c
    uint32_t step0 = (uint32_t)(challenge + ((challenge >> 32) * 8)) ^ 0xb2c;
    // step1 (high32(challenge) - 32(challenge >> 16)) * 5 + 0x59cc, then cut to 32 bits
    uint32_t step1 = (uint32_t)((uint64_t)((uint32_t)(challenge >> 32) - (uint32_t)(challenge >> 16)) * 5 + 0x59cc);
    // step2 (rdi + rdi * 2 + esi) ^ 0x24bb
    uint32_t step2 = (step1 * 3 + (uint32_t)(challenge)) ^ 0x24bb;
    values[count-1] = step2 ^ step0;
    return write_answer(out, cap, values, count);
}
```

验证（seed 114514）：

```
$ ./defuser --seed 114514 --phase 1 | ./bomb --seed 114514 --phase 1
Phase 1 challenge: 0xc3e450079c288d47
Input (unsigned decimal integers):
Phase 1 defused.
```

Phase 2

用开头的方法看出 count 是 6。`value[0]` 可以从以下汇编恢复逻辑

```
40215a:  mov    %r13d,%eax         // r13d is the lower 32 bits of challenge, assigned to eax.
40215d:  mov    -0x40(%rbp),%esi   // esi is value[0] which is first input
402160:  movzbl %ah,%eax           // eax = (eax >> 8) & 0xff, second lowest byte of challenge.
402163:  lea    (%rax,%rax,2),%eax // eax = eax * 3, no chance of overflow.
402166:  cmp    %eax,%esi          // compare value[0] with 3 * second lowest byte of challenge.
```

后面是一个循环逻辑，`edx` 作为 counter，每次对 challenge 进行一些 i 有关的计算，包括还要异或上一个结果，这些计算出来的结果和当前的 `values[i]` 比较。循环逻辑如下

```
40216a:  mov    $0x1,%edx
40216f:  lea    0xa(%rdx),%ecx           // ecx = edx + 10 = 11.
402172:  mov    %r13,%rax                // rax = challenge
402175:  shr    %cl,%rax                 // cl is the lowerst 8 bits of ecx, so rax = challenge >> 11
402178:  and    $0x3f,%eax               // eax = (challenge >> 11) & 0x3f
40217b:  add    %edx,%eax                // eax += i
40217d:  xor    %esi,%eax                // eax ^= value[i-1]
40217f:  mov    -0x40(%rbp,%rdx,4),%esi  // esi = value[i]
402183:  lea    (%rax,%rax,4),%eax       // eax = eax * 5
402186:  cmp    %eax,%esi                // if eax != value[i], return false.
402188:  jne    402115 <phase_02+0x25>
40218a:  add    $0x1,%rdx                // rdx+=1
40218e:  cmp    $0x6,%rdx                // if rdx == 6, done, else to next iteration
```

容易根据此恢复出循环逻辑，下面是代码截图

```c
int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    uint32_t x;
    (void)challenge;
    // TODO: 根据 phase_02 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    count = 6;

    values[0] = 3 * ((uint32_t)(challenge >> 8) & 0xff);
    int i = 1;
    for (; i < 6; ++i) {
        x =  ((((uint32_t) (challenge >> (i + 10))) & 0x3f) + i) ^ values[i-1];
        x = x + (x << 2);
        values[i] = x;
    }
    return write_answer(out, cap, values, count);
}
```

验证（seed 114514）：

```
$ ./defuser --seed 114514 --phase 2 | ./bomb --seed 114514 --phase 2
Phase 2 challenge: 0xed3459d3bab69fb5
Input (unsigned decimal integers):
Phase 2 defused.
```

Phase 3

phase_03 先看出 count 是 2，然后先得到 `values[0]`

```
4022dc:  mov    %r12d,%edi         // edi is the low 32 bits of r12, which is challenge.
4022df:  xor    %eax,%eax
4022e1:  and    $0x7,%edi          // edi & 0x7.
4022e4:  cmp    %edi,-0x18(%rbp)   // compare edi with values[0], values[0] thus = challenge & 0x7.
```

然后把 challenge 计算后的结果传入 `p3_dispatch`

```
4022dc:  mov    %r12d,%edi         // edi is the low 32 bits of r12, which is challenge.
4022df:  xor    %eax,%eax
4022e1:  and    $0x7,%edi          // edi & 0x7, the 1st argument of p3_dispatch (the switch value).

4022e9:  mov    %r12,%rax          // rax = challenge
4022ec:  mov    %r12,%rsi          // rsi = challenge
4022ef:  shr    $0x3,%rax          // rax = challenge >> 3
4022f3:  shr    $0x20,%rsi         // rsi = challenge >> 32
4022f7:  xor    %eax,%esi          // esi = (challenge >> 32) ^ (challenge >> 3), the 2nd argument.
4022f9:  call   402230 <p3_dispatch>
4022fe:  cmp    %eax,-0x14(%rbp)   // return value is thus values[1].
```

`p3_dispatch` 的逻辑是一个 switch case，输入的一个数作为的条件调用不同的函数，这里我们需要 dump 出每个地址对应的指令地址，跳转到对应的 case

```
40223d:  notrack jmp *0x405820(,%rax,8)   // 跳到 table[eax] 里存的地址
```

跳转表在 `.rodata` 的 `0x405820`，共 8 项，每项 8 字节：

```
$ gdb -batch -ex "x/8gx 0x405820" ./bomb
0x405820:  0x0000000000402258  0x0000000000402260
0x405830:  0x0000000000402268  0x0000000000402270
0x405840:  0x0000000000402278  0x0000000000402280
0x405850:  0x0000000000402248  0x0000000000402250
```

把表项对应到 case 函数，表里的地址都指向 `p3_dispatch` 中间的一行 `jmp`：

```
402248:  jmp    4021c0 <p3_case_2>
402250:  jmp    402200 <p3_case_6>
402258:  jmp    4021d0 <p3_case_3>
402260:  jmp    402210 <p3_case_7>
402268:  jmp    4021a0 <p3_case_0>
402270:  jmp    4021b0 <p3_case_1>
402278:  jmp    4021f0 <p3_case_5>
402280:  jmp    4021e0 <p3_case_4>
```

后面每个 case 具体的逻辑就省略了，比较简单，可以直接看代码

```
57  int solve_phase_3(uint64_t challenge, char *out, size_t cap) {
58      uint32_t values[MAX_FIELDS] = {0};
59      size_t count = 0;
60      (void)challenge;
61      // TODO: 根据 phase_03 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
62      count = 2;
63      values[0] = (uint32_t) challenge & 0x7;
64      uint32_t b = (challenge >> 3) ^ (challenge >> 32);
65      switch (values[0]) {
66          case 0: values[1] = 0x12dd1 + 5 * b; break;
67          case 1: values[1] = (b >> 2) + (b ^ 0x6c59); break;
68          case 2: values[1] = (b >> 1) + (b ^ 0x4177); break;
69          case 3: values[1] = (b * 2) ^ 0x34ee; break;
70          case 4: values[1] = 5 * b + 0xe015; break;
71          case 5: values[1] = b * 2 ^ 0x6504; break;
72          case 6: values[1] =  0x77ab + 5 * b; break;
73          case 7: values[1] = 0xee9d + 5 * b; break;
74          default: values[1] = 0; break;
75      }
76      // a = 0 - 6 corresponds to p3 case 3 7 0 1 5 4 2 6
77
78      return write_answer(out, cap, values, count);
79  }
```

验证（seed 114514）：

```
$ ./defuser --seed 114514 --phase 3 | ./bomb --seed 114514 --phase 3
Phase 3 challenge: 0x6e51758b9acd24fd
Input (unsigned decimal integers):
Phase 3 defused.
```

Phase 4

phase_04 考察的是一个递归，这个有点难。首先 count 是 1，我们先找到递归函数传入的四个变量

```
4023c8:  mov    %r12,%rcx
4023cb:  mov    %r12,%rax
4023ce:  mov    %r12d,%edi         // rcx , rax , edi set to challenge.
4023d1:  xor    %esi,%esi          // esi set to 0
4023d3:  shr    $0x20,%rax         // challenge >> 32
4023d7:  shr    $0x7,%rcx          // challenge >> 7
4023db:  and    $0x7f,%edi         // challenge & 0x7f
4023de:  mov    $0x7f,%edx         // edx set to 0x7f
4023e3:  xor    %eax,%ecx          // ecx = (challenge >> 32) ^ (challenge >> 7)
```

可以看出这四个变量分别是

```
uint32_t s = 0;
uint32_t di = challenge & 0x7f;
uint32_t dx = 0x7f;
uint32_t c = (challenge >> 32) ^ (challenge >> 7);
```

然后进入递归逻辑 `p4_walk`。首先算出了新的 di，然后可以看出终止条件是旧的 di 等于新的 di。然后也容易看出这就是算一个二分。

```
402314:  mov    %edi,%eax          // eax = edi
402316:  mov    %edx,%edi          // edi = edx = 0x7f
402318:  sub    %esi,%edi          // edi = 0x7f - esi, = 0x7f - 0 in the first call.
40231a:  shr    $1,%edi            // edi >> 1
40231c:  add    %esi,%edi          // edi = (0x7f - si) >> 1 + si
40231e:  cmp    %eax,%edi          // compare eax with edi, edi is (edx - esi) >> 1 + esi, eax = former
edi.
402320:  je     402370 <p4_walk+0x60>  // go out of recursion if eax == edi
```

这里可以看出 `edi = (edx - esi) >> 1 + esi`，也就是取了递归变量 `edi` 和 `esi` 的中间值作为新的 `edi`，如果这个新的 value 正好是 di 就终止（`je` 跳到 `402370` 的递归出口），否则继续递归，直到现在基本能看出是一个二分查找，输入的 s, di, dx, c 的作用分别是开始，目标，结束，和一个用于计算返回的辅助数。这里要注意调用后逻辑，也就是你 return 的东西应该是一个递归调用的返回值然后和 c 进行了一些机选，比如

```
402370:  lea    0x20(%rdi),%eax    // eax = 0x20 + edi
402373:  xor    %ecx,%eax          // return eax ^ ecx
```

这个就是递归退出的后计算。

```
402330:  jb     402350 <p4_walk+0x40>  // if eax < edi, go to 402350
```

这个可以看出递归的分支，然后就是计算新的传入参数，调用，然后后计算。例如

```
402350:  lea    -0x1(%rdi),%edx    // edx = edi - 1
402353:  mov    %eax,%edi          // edi = eax
402355:  call   402310 <p4_walk>
40235a:  lea    (%rax,%rax,2),%eax
40235d:  xor    %ebx,%eax
```

另一个分支这里就省略了，最后翻译出来的函数如下

```
uint32_t walk_phase_4(uint32_t s, uint32_t di, uint32_t dx, uint32_t c) {
    uint32_t a = di;
    uint32_t newdi = ((dx - s) >> 1) + s;
    if (a == newdi) {
        return (0x20 + newdi) ^c;
    } else if (a < newdi) {
        return (walk_phase_4(s, a, newdi - 1, c) * 3) ^ (newdi + c) ^ 0x3;
    } else {
        return (walk_phase_4(newdi + 1, a, dx, c) * 3) ^ (newdi + c) ^ 0xf;
    }
}

int solve_phase_4(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    (void)challenge;
    // TODO: 根据 phase_04 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    count = 1;
    uint32_t s = 0;
    uint32_t di = challenge & 0x7f;
    uint32_t dx = 0x7f;
    uint32_t c = (challenge >> 32) ^ (challenge >> 7);

    values[0] = walk_phase_4(s, di, dx, c);
    return write_answer(out, cap, values, count);
}
```

验证（seed 114514）：

```
$ ./defuser --seed 114514 --phase 4 | ./bomb --seed 114514 --phase 4
Phase 4 challenge: 0x077b3d42a7534dc6
Input (unsigned decimal integers):
Phase 4 defused.
```

Phase 5

首先看出 count 是 2。这个题有点难。`p5_traverse` 函数传入 challenge 和一个 out 指针，然后进入一个比较复杂的逻辑，先提取 challenge 最后 3 位，我们这里记作 a，后面可以看出这个 a 用于 index 遍历链表的时候的开头

```
40240d:  and    $0x7,%eax                // challenge & 0x7 -> a
402410:  shr    $0x3,%rcx                // challenge >> 3
402414:  mov    0x405b60(,%rax,8),%rdx   // *(0x405b60 + a*8) -> rdx
```

这里暗示了 `0x405b60` 开始存了一个表映射 index 和 node 结构体，来确定遍历开头的 node，我们用 gdb 找到这个表

```
$ gdb -batch -ex "x/8gx 0x405b60" ./bomb
0x405b60 <p5_heads>:    0x0000000000405c10      0x0000000000405c00
0x405b70 <p5_heads+16>: 0x0000000000405bd0      0x0000000000405bc0
0x405b80 <p5_heads+32>: 0x0000000000405be0      0x0000000000405c30
0x405b90 <p5_heads+48>: 0x0000000000405c20      0x0000000000405ba0
```

可以看出这 8 个地址每个对应一个 node 结构体我们猜一下最小的在这个 `405ba0`，从这个里面 dump 出来一些可以发现他打了标签叫做 `p5_nodes`，每一个 node 结构体由下一个 node 地址，和两个 `uint32_t` 的 value 组成，然后我们使用一个 gdb 命令清晰的看出所有 node

```
$ gdb -batch -ex "x/48wx 0x405ba0" ./bomb
0x405ba0 <p5_nodes>:     0x00405bf0  0x00000000  0x0000087b  0x0000d973
0x405bb0 <p5_nodes+16>:  0x00000000  0x00000000  0x00000a86  0x00005722
0x405bc0 <p5_nodes+32>:  0x00405be0  0x00000000  0x00000b64  0x0000fed3
0x405bd0 <p5_nodes+48>:  0x00405bc0  0x00000000  0x00000684  0x0000268d
0x405be0 <p5_nodes+64>:  0x00405c30  0x00000000  0x0000034f  0x00000505
0x405bf0 <p5_nodes+80>:  0x00405c50  0x00000000  0x00000011  0x0000af26
0x405c00 <p5_nodes+96>:  0x00405bd0  0x00000000  0x00000474  0x0000085f
0x405c10 <p5_nodes+112>: 0x00405c00  0x00000000  0x00000255  0x0000113d
0x405c20 <p5_nodes+128>: 0x00405ba0  0x00000000  0x00000549  0x000032e4
0x405c30 <p5_nodes+144>: 0x00405c20  0x00000000  0x00000763  0x00005cf0
0x405c40 <p5_nodes+160>: 0x00405bb0  0x00000000  0x0000016e  0x0000d1e4
0x405c50 <p5_nodes+176>: 0x00405c40  0x00000000  0x0000093a  0x00003c66
```

这里我们就能恢复出一个 nodes 列表和 heads 的 index 和 node 的对应关系。然后回到 `p5_traverse` 函数的汇编代码。

```
40241c:  mov    %rdi,%rax
40241f:  shr    $0x20,%rdi         // challenge >> 32
402423:  shr    $0x6,%rax          // challenge >> 6
402427:  mov    0x8(%rdx),%esi     // *(rdx + 8) -> esi
40242a:  xor    %edi,%eax          // rax = (challenge >> 6) ^ (challenge >> 32)
40242c:  xor    0xc(%rdx),%eax     // eax ^= *(rdx + 0xc)
40242f:  mov    %ecx,%edi          // edi = challenge >> 3
402431:  xor    $0xd322,%eax       // eax ^= 0xd322
402436:  mov    %esi,(%r8)         // esi is stored at address r8
402439:  add    %esi,%eax          // eax += esi
40243b:  and    $0x3,%edi          // edi &= 0x3
```

显然这里 `esi` 就是 `heads[a]` 这个 node 的 v1，然后 `*(rdx + 0xc)` 这个就是 v2，然后回溯可以知道 `r8` 是 `phase_05` 里局部变量 out（`-0x14(%rbp)`）的地址，`phase_05` 最后会把它和第 2 个输入 `values[1]`（`-0x18(%rbp)`）比较，目前把 `esi` 暂存上去。这一段就是对 a 进行了一系列计算。然后 `edi` 作为后续循环的次数。

```
40243e:  je     40245a <p5_traverse+0x5a>  // jump if edi == 0
402440:  mov    $0x1,%ecx          // as a counter
402445:  mov    (%rdx),%rdx        // rdx = *(rdx) to next of the linked list
402448:  add    $0x1,%ecx          // ecx += 1
40244b:  mov    0x8(%rdx),%esi     // esi = *(rdx + 8)
40244e:  xor    0xc(%rdx),%eax     // eax ^= *(rdx + 0xc)
402451:  add    %esi,%eax          // eax += esi
402453:  mov    %esi,(%r8)         // store esi at address r8
402456:  cmp    %ecx,%edi          // compare ecx and edi
402458:  jae    402445 <p5_traverse+0x45>  // next iteration if edi >= ecx
```

这就是循环的逻辑，`edi` 是循环次数，使用 `rdx = *(rdx)` 遍历链表，对 a 和 `esi` 进行更新，每次把新的 `esi` 存到 out 上（所以 out 最后是遍历到的最后一个 node 的 v1）。最后返回 `eax`。最后 `values[0]` 就是 a，然后 `values[1]` 就是最后的 `esi`。

代码截图如下

```c
struct node{
    int next;
    uint32_t v1;
    uint32_t v2;
};

static const struct node nodes[] = {
    {5, 0x0000087b, 0x0000d973},
    {-1, 0x00000a86, 0x00005722},
    {4, 0x00000b64, 0x0000fed3},
    {2, 0x00000684, 0x0000268d},
    {9, 0x0000034f, 0x00000505},
    {11, 0x00000011, 0x0000af26},
    {3, 0x00000474, 0x0000085f},
    {6, 0x00000255, 0x0000113d},
    {0, 0x00000549, 0x000032e4},
    {8, 0x00000763, 0x00005cf0},
    {1, 0x0000016e, 0x0000d1e4},
    {10, 0x0000093a, 0x00003c66},
};

static const int heads[]= {
    7, 6, 3, 2, 4, 9, 8, 0,
};

int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    (void)challenge;
    // TODO: 根据 phase_05 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    count = 2;
    uint32_t a = challenge & 0x7;
    uint32_t times = (challenge >> 3) & 0x3;
    uint32_t i = 1;
    int head = heads[a];
    uint32_t s = nodes[head].v1;
    a = nodes[head].v2 ^ ((challenge >> 6) ^ (challenge >> 32)) ^ 0xd322;
    a += s;
    values[1] = s;
    if (times > 0) {
        for (; i <= times; ++i) {
            head = nodes[head].next;
            s = nodes[head].v1;
            a ^= nodes[head].v2;
            a += s;
            values[1] = s;
        }
    }
    values[0] = a;
    return write_answer(out, cap, values, count);
}
```

验证（seed 114514）：

```
$ ./defuser --seed 114514 --phase 5 | ./bomb --seed 114514 --phase 5
Phase 5 challenge: 0xfcac85f876912d1c
Input (unsigned decimal integers):
Phase 5 defused.
```

Phase 6

首先 count 是 1，`p6_dispatch` 这个函数里面先根据 challenge 算几个数

```
40256c:  shr    $0x6,%rdi          // challenge >> 6
402570:  mov    %rbx,%rax
402573:  shr    $0x20,%rax         // challenge >> 32
402577:  xor    %eax,%edi          // edi = (challenge >> 6) ^ (challenge >> 32)
402579:  mov    %rbx,%rax          // rax = challenge
40257c:  sub    $0x8,%rsp
402580:  and    $0x38,%ebx         // ebx = challenge & 0x38
402583:  and    $0x7,%eax          // eax = challenge & 0x7
402586:  call   *0x405e00(,%rax,8)
```

最后这个 call 最重要，含义是调用一个函数指针数组，数组的地址是 `0x405e00`，数组的索引是 `challenge & 0x7`。传入的变量是 `edi`，因此我们要找到这个地址附近存的函数地址，用 gdb 就可以恢复出数组 index 和每个函数的对应关系。

```
$ gdb -batch -ex "x/8a 0x405e00" ./bomb
0x405e00 <p6_ops>:      0x402510 <p6_op_2>      0x4024f0 <p6_op_1>
0x405e10 <p6_ops+16>:   0x402530 <p6_op_4>      0x4024f0 <p6_op_1>
0x405e20 <p6_ops+32>:   0x402540 <p6_op_5>      0x4024f0 <p6_op_1>
0x405e30 <p6_ops+48>:   0x402520 <p6_op_3>      0x4024d0 <p6_op_0>
```

然后再 dump 出每个函数分别是啥就行。后续还把第一次调用的返回值计算一下传入第二次调用的变量，第二次调用的函数被 `rbx` indexing

```
40258d:  xor    $0xcb1f,%eax
402592:  mov    %eax,%edi
402594:  call   *0x405e00(%rbx)
40259a:  mov    -0x8(%rbp),%rbx
40259e:  leave
40259f:  add    $0xa499,%eax
```

代码如下

```c
typedef uint32_t (*p6_op)(uint32_t);

static uint32_t p6_op_0(uint32_t a) { return (a ^ (a >> 7)) + 0x5a06; }
static uint32_t p6_op_1(uint32_t a) { return (a * 5) ^ (0xe3fd + (a << 7)); }
static uint32_t p6_op_2(uint32_t a) { return (a + 0x31a9) ^ (a >> 2); }
static uint32_t p6_op_3(uint32_t a) { return  ((a << 5) | (a >> 27)) ^ 0x8661; }
static uint32_t p6_op_4(uint32_t a) { return ((a >> 2) | (a << 30)); }
static uint32_t p6_op_5(uint32_t a) { return (a << 5) + (a ^ 0x6a58); }

static const p6_op p6ops[] = {
    p6_op_2, p6_op_1,
    p6_op_4, p6_op_1,
    p6_op_5, p6_op_1,
    p6_op_3, p6_op_0,
};

int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    (void)challenge;
    // TODO: 根据 phase_06 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    count = 1;
    uint32_t a = (challenge >> 6) ^ (challenge >> 32);
    uint32_t b = challenge & 0x38;
    uint32_t c = challenge & 0x7;
    // 0x405e00 <p6_ops>:      0x402510 <p6_op_2>      0x4024f0 <p6_op_1>
    // 0x405e10 <p6_ops+16>:   0x402530 <p6_op_4>      0x4024f0 <p6_op_1>
    // 0x405e20 <p6_ops+32>:   0x402540 <p6_op_5>      0x4024f0 <p6_op_1>
    // 0x405e30 <p6_ops+48>:   0x402520 <p6_op_3>      0x4024d0 <p6_op_0>
    uint32_t first = p6ops[c](a);
    first ^= 0xcb1f;
    uint32_t second = p6ops[b >> 3](first);
    values[0] = second + 0xa499;
    return write_answer(out, cap, values, count);
}
```

验证（seed 114514）：

```
$ ./defuser --seed 114514 --phase 6 | ./bomb --seed 114514 --phase 6
Phase 6 challenge: 0x2055abe6336eebb1
Input (unsigned decimal integers):
Phase 6 defused.
```

Secret Phase

secret_phase 其实不难，已经提示了 context 是根据之前六个算的。首先我们要看出 secret phase 验证的时候在 bomb 里的调用链。所以我们需要先看 main。

```
40162f:  call   402810 <transcript_context>
401634:  mov    $0x7,%esi
401639:  lea    -0x8c0(%rbp),%rdi
401640:  mov    %rax,%rbx          // rbx is the returned context
401643:  call   402c40 <phase_challenge>
401648:  mov    %rbx,%rdx          // rdx is the context
40164b:  lea    -0x730(%rbp),%rdi  // the input array
401652:  mov    %rax,%rsi          // rsi is the secret phase challenge
401655:  call   402710 <secret_gate>
```

这里计算出了 context 以后，把 context，输入的那一行和 challenge 输入到 secret gate，然后我们 dump secret gate。这个函数先开了一个局部数组 buffer，并且检查了输入 array 的长度

```
40272b:  lea    -0xd0(%rbp),%r12   // buffer

402749:  mov    %rbx,%rdi          // array pointer
40274c:  call   401190 <strlen@plt>
402751:  sub    $0x24,%rax         // strlen - 36
402755:  cmp    $0x7f,%rax
402759:  jbe    402770 <secret_gate+0x60>  // strlen <= 163 >= 36
```

然后的逻辑是把 array 和 buffer 传入 secret copy，进去看一下其实就是把 array copy 到 buffer。然后检验了 32-36 个 char 是 OPEN

```
402776:  call   4026f0 <secret_copy>
40277b:  cmpl   $0x4e45504f,-0xb0(%rbp)    // line[32-35] == "OPEN"
```

然后后面就是常规逻辑，把 buffer 里 [36] 对应的地址（复制后的那一行从第 36 个字符开始的部分），context 和 challenge 传入 `secret_phase` 这个函数，后面就是正常的计算逻辑，这里省略了

```
402787:  mov    %r14,%rdx          // context
40278a:  mov    %r13,%rsi          // challenge
40278d:  lea    -0xac(%rbp),%rdi   // rbp - 172 = &buf[36], buffer starts at rbp - 0xd0
```

最后恢复出来代码如下，注意这里我们知道前面 32 个 char 随便写，然后后面四个是 OPEN，然后跟空格分割的数字，这里计算逻辑的汇编我们都省略了，一点一点恢复计算逻辑就行

```c
#define MAX_FIELDS 8
#define SECRET_PREFIX "I can't live without claude codeOPEN"

int solve_secret(uint64_t challenge, uint64_t context, char *out, size_t cap) {
    (void)challenge;
    (void)context;
    (void)out;
    (void)cap;
    // TODO（选做）：根据 secret_gate、secret_copy 和 secret_phase 的汇编构造完整的第七行，写入 out 后返回 0。
    // 第七行不一定只有整数，可以直接用 snprintf 写入；还没完成时保持返回 1，测试会显示 not attempted。
    uint32_t a = context & 3;
    uint32_t tmp1 = a * 0x53b8;
    uint32_t tmp2 = challenge ^ (challenge >> 32);
    uint32_t tmp3 = context ^ (context >> 32);
    uint32_t tmp4 = tmp1 ^ (context >> 16);
    uint32_t mixinput = tmp2 ^ tmp3;
    uint32_t mixoutput =  ((mixinput >> 3)  ^ 0xe24e) + (mixinput ^ 0x53b8);
    uint32_t b = tmp4 ^ mixoutput;
    uint32_t c = b ^ (tmp3 + a + 0xe24e + (tmp2 >> 3));
    int n = snprintf(out, cap, SECRET_PREFIX "%" PRIu32 " %" PRIu32 " %" PRIu32, a, b, c);
    if (n < 0 || (size_t)n >= cap) return 1;
    return 0;
}
```

验证（seed 114514）：

```
$ { ./defuser --seed 114514; ./defuser --secret --seed 114514; } | ./bomb --seed 114514
Phase 1 challenge: 0xc3e450079c288d47
Phase 1 defused.
Phase 2 challenge: 0xed3459d3bab69fb5
Phase 2 defused.
Phase 3 challenge: 0x6e51758b9acd24fd
Phase 3 defused.
Phase 4 challenge: 0x077b3d42a7534dc6
Phase 4 defused.
Phase 5 challenge: 0xfcac85f876912d1c
Phase 5 defused.
Phase 6 challenge: 0x2055abe6336eebb1
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
```

（省略了剧情文字，只保留关键输出。第 7 行为 `I can't live without claude codeOPEN1 3156679833 1798710449`。）

参考资料

课程实验文档 https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/

AI 声明

本次试验中我使用了 ai 问一些有关某些汇编什么意思，以及 gdb objdump 这些工具怎么用的问题，其余所有逻辑都是我自己推导出来的。此外 AI 还润色了实验报告，帮助我截图对应的代码和排版。实验报告的内容和思路也是我写的。